

Архиватор на основе алгоритма Хаффмана

Долженко Александра Борисовна

Группа: 25.Б42-мм

Кафедра: системного программирования

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Алгоритм Хаффмана — метод сжатия без потерь, основанный на построении оптимального префиксного кода. Применяется в JPEG, MP3, gzip. В рамках практики разработано консольное приложение для сжатия и восстановления файлов, что позволяет освоить бинарные деревья, битовые потоки и управление памятью в C.

2. Постановка задачи

Целью работы является получение практического опыта низкоуровневого программирования на языке C путём реализации консольного приложения для сжатия и восстановления файлов по алгоритму Хаффмана.

Для достижения цели требуется решить следующие задачи:

1. Выполнить обзор алгоритма Хаффмана и существующих методов сжатия без потерь.
2. Спроектировать архитектуру приложения и формат сжатого файла.
3. Реализовать сжатие и распаковку файлов.
4. Провести эксперименты по оценке степени сжатия и производительности.

3. Обзор

Алгоритм Хаффмана предложен Дэвидом Хаффманом в 1952 году [1]. Он строит префиксный код на основе частот символов: чем чаще символ, тем короче его код. Существуют и другие методы сжатия без потерь: LZ77 [2], LZW, арифметическое кодирование, RLE. Хаффман уступает им по степени сжатия, но проще в реализации и быстрее. В gzip [3] он используется вместе с LZ77.

Вывод: алгоритм Хаффмана не является лучшим по степени сжатия, но все равно широко применяется на практике в современных архиваторах в сочетании с другими методами сжатия, а также остаётся классическим учебным примером.

4. Архитектура и технические детали

Программа состоит из четырёх модулей:

- **bitOperation** — работа с битовым буфером (запись/чтение битов);
- **queue** — очередь с приоритетом (куча) для построения дерева;
- **huffmanTree** — построение дерева и генерация кодов;
- **huffman** — основная логика сжатия и распаковки.

Для построения дерева используется бинарная куча со сложностью $O(\log n)$ на операцию. Отдельно обрабатывается случай одного уникального символа: создаётся фиктивный родительский узел.

4.1. Обоснование выбора представления таблицы частот

Таблица хранится в виде пар «символ (1 байт) — частота (8 байт)». У данной реализации есть альтернативы, например: полная таблица из 256 частот и канонические коды Хаффмана [4].

Преимущества: для текстов (70–90 уникальных символов, что составляем примерно один язык) заголовков занимает 630–810 байт вместо 2048;

Ограничения: для полного алфавита заголовков занимает 2304 байта, что больше фиксированной таблицы из 256 частот (2048 байт); при сжатии файлов менее 1 КБ они наоборот увеличиваются из-за размера заголовка.

Возможное улучшение: выбор формата через маркер (0xFF — моя реализация через пары, 0xFE — полная таблица, 0xFD — однобайтовые коды) и тд.

5. Эксперименты

5.1. Методика

Использовались файлы различных типов: тексты, повторяющиеся данные, нулевые данные, смешанные данные, изображения JPEG, бинарные файлы. Для замеров времени выполнялось 10 прогонов; вычислялись матожидание и СКО.

5.2. Степень сжатия

Степени сжатия приведены в таблице 1.

Таблица 1: Результаты сжатия тестовых файлов

Файл	Исходный (В)	Сжатый (В)	Экономия (%)
Текстовые данные			
text_en_dict_1MB.txt	985,084	551,749	43.99
text_en_dict_100k.txt	102,400	60,417	41.00
text_en_dict_10k.txt	10,240	5,950	41.89
text_58b.txt	58	251	-332.76
Повторяющиеся данные			
repeated_10MB.txt	10,485,760	3,713,782	64.58
repeated_50k.txt	42,000	16,594	60.50
single_char_10K.txt	10,000	1,281	87.19
Нулевые данные			
zero_10MB.bin	10,485,760	1,310,742	87.50
zero_100k.bin	102,400	12,822	87.48
zero_10k.bin	10,240	1,302	87.29
Изображения			
Image_450k.JPG	454,850	451,021	0.84
Image_14MB.jpg	14,568,090	14,547,943	1.00
Бинарные файлы			
bash_binary.bin	1,446,024	1,128,089	21.99
mixed_10MB.bin	10,485,760	8,880,266	15.32
mixed_10K.bin	5,120	7,434	-45.19
random_100k.bin	102,400	104,717	-2.26
random_10k.bin	10,240	12,556	-22.61

Результаты. Эффективность сжатия зависит от энтропии данных:

- **Высокая степень сжатия** достигается на данных с низкой энтропией: нулевые данные (до 87.50%), повторяющиеся данные (60–87%), тексты (41–45%).
- **Умеренная степень сжатия** наблюдается на исполняемых файлах (bash_binary.bin: 21.99%): машинный код содержит много нулей (выравнивание секций), повторяющихся инструкций и текстовых строк, однако энтропия выше, чем у текста, а число уникальных символов приближается к 256.
- **Низкая степень сжатия** характерна для уже сжатых данных (Image_14MB.jpg: 1.00%).
- **Увеличение размера** наблюдается на файлах со случайными данными (random_100k.bin: –2.26%), а так же на очень маленьких файлах (text_58b.txt: –332.76%), так как заголовок превышает объём самих данных до сжатия.

Таким образом, представление таблицы частот в виде пар «символ — частота» оптимально для текстовых данных, но уступает фиксированной таблице на данных с полным алфавитом и высокой энтропией.

5.3. Конфигурация устройства

Замеры производительности проводились на следующей конфигурации виртуальной машины:

- **Процессор:** 12th Gen Intel Core i7-1280P;
- **Доступно ядер:** 4;
- **Оперативная память:** 7.8 ГБ;
- **Накопитель:** VBOX HARDDISK, 50 ГБ;
- **Операционная система:** Ubuntu 24.04.3 LTS;
- **Ядро:** Linux 7.0.0-31-generic;
- **Файловая система:** ext4;
- **Компилятор:** GCC 13.3.0;
- **Флаги компиляции:** -Wall -Wextra -Wpedantic -O0;
- **CMake:** 3.28.3;
- **Среда:** VirtualBox.

5.4. Производительность

Результаты замеров времени приведены в таблице 2.

Таблица 2: Время сжатия файлов (N = 10 прогонов), время указано как матожидание ± СКО.

Файл	Размер (В)	Время, сек
Текстовые данные		
text_en_dict_1MB.txt	985,084	0.215 ± 0.027
text_en_dict_100k.txt	102,400	0.029 ± 0.005
text_en_dict_10k.txt	10,240	0.011 ± 0.003
text_58b.txt	58	0.006± < 0.001
Повторяющиеся данные		
repeated_10MB.txt	10,485,760	0.459 ± 0.025
repeated_50k.txt	42,000	0.008 ± 0.001
single_char_10K.txt	10,000	0.005± < 0.001
Нулевые данные		
zero_10MB.bin	10,485,760	0.212 ± 0.023
zero_100k.bin	102,400	0.008 ± 0.001
zero_10k.bin	10,240	0.007 ± 0.001
Смешанные данные		
mixed_10MB.bin	10,485,760	8.157 ± 0.251
mixed_10K.bin	5,120	0.011 ± 0.002
Изображения		
Image_14MB.jpg	14,568,090	10.077 ± 0.559
Image_450k.JPG	454,850	0.364 ± 0.010
Бинарные файлы		
bash_binary.bin	1,446,024	0.944 ± 0.040
random_100k.bin	102,400	0.085 ± 0.003
random_10k.bin	10,240	0.015 ± 0.003

Выводы: время сжатия растёт с размером файла и его энтропией. Наибольшая скорость налюдается на нулевых данных (10 МВ за 0.21 сек, 50 МВ/сек) и повторяющихся данных (10 МВ за 0.46 сек, 23 МВ/сек). Наименьшая — на случайных данных и больших JPEG (10 МВ за 7-8 сек, 1.3 МВ/сек) из-за глубокого дерева и высокой энтропии.

5.5. Проверка памяти

Valgrind не обнаружил утечек памяти:

```
1 ==13166== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
2 ==13166== All heap blocks were freed -- no leaks are possible
```

6. Заключение

В ходе работы получены следующие результаты:

- Реализован архиватор на основе алгоритма Хаффмана: сжатие и распаковка файлов, поддержка любых типов данных, обработка краевых случаев.
- Эксперименты показали: наилучшее сжатие достигается на данных с низкой энтропией (до 87.5%), текст сжимается на 41-44%, уже сжатые данные не сжимаются.
- Измерена производительность: скорость составляет от 1.3 МВ/сек (случайные данные) до 50 МВ/сек (нулевые данные), утечек памяти не обнаружено.

Реализацию можно найти по адресу: https://github.com/DolzhenkoAlexa/Project_huffman (дата обращения: 23 сентября 2026 г.).

Список литературы

[1] Huffman coding. URL: https://en.wikipedia.org/wiki/Huffman_coding (дата обращения: 23 сентября 2026 г.).

[2] LZ77 and LZ78. URL: https://en.wikipedia.org/wiki/LZ77_and_LZ78 (дата обращения: 23 сентября 2026 г.).

[3] GNU Gzip. URL: <https://www.gnu.org/software/gzip/> (дата обращения: 23 сентября 2026 г.).

[4] Canonical Huffman Code. URL: https://en.wikipedia.org/wiki/Canonical_Huffman_code (дата обращения: 23 сентября 2026 г.).